

ส่วนที่ 1: หลักการออกแบบ

1. SOLID Principles

ใน Lab 8 มีการนำหลัก SOLID มาใช้ในการออกแบบระบบ

เพื่อให้โปรแกรมเป็นระเบียบ แก้ไขง่าย

และสามารถเพิ่มเติมความสามารถในอนาคตได้

S — Single Responsibility Principle (SRP)

แต่ละ Class มีหน้าที่ของตัวเองอย่างชัดเจน เช่น Product ใช้เก็บข้อมูลหลักของสินค้า,

ProductDetail ใช้เก็บรายละเอียดเพิ่มเติมของสินค้า และ Review

ใช้เก็บข้อมูลความคิดเห็นของลูกค้า นอกจากนี้ยังแยก Controller, Service และ Repository ออกจากกัน ทำให้แต่ละส่วนไม่ทำงานซ้ำหน้าที่กัน

O — Open/Closed Principle (OCP)

ระบบออกแบบให้สามารถเพิ่มความสามารถใหม่ได้โดยไม่ต้องแก้ไขโค้ดเดิมมาก

เช่น สามารถเพิ่มรูปแบบส่วนลดใหม่หรือเพิ่มข้อมูลที่เกี่ยวข้องกับสินค้าได้

โดยไม่จำเป็นต้องรวมทุกอย่างไว้ใน Product

L — Liskov Substitution Principle (LSP)

Strategy แต่ละประเภทสามารถใช้งานแทนกันได้ เพราะมีโครงสร้างพื้นฐานเดียวกัน

เช่น ส่วนลดสมาชิกและส่วนลดเทศกาลสามารถนำมาใช้แทน Strategy ปกติได้

I — Interface Segregation Principle (ISP)

แยก Repository ตามหน้าที่ของแต่ละ Entity เช่น ProductRepository,

ProductDetailRepository และ ReviewRepository ทำให้แต่ละ Interface มีหน้าที่เฉพาะ

ไม่รวมทุกอย่างไว้ใน Interface ขนาดใหญ่เพียงตัวเดียว

D — Dependency Inversion Principle (DIP)

Service ไม่ควรผูกกับการสร้าง Repository โดยตรง แต่ให้ทำงานผ่าน Repository Interface

และใช้ Constructor Injection เพื่อให้ Spring จัดการ

Dependency ทำให้ระบบมีความยืดหยุ่นและทดสอบได้ง่ายขึ้น

SOLID เกี่ยวข้องกับ Entity อย่างไร

การแยก Product, ProductDetail และ

Review ออกจากกัน เป็นการออกแบบให้ข้อมูลแต่ละประเภทมีความรับผิดชอบของตัวเอง

ตัวอย่างเช่น ไม่ควรนำข้อมูลรายละเอียดทั้งหมด เช่น น้ำหนัก การรับประกัน

และขนาดสินค้า มารวมไว้ใน Product

หากข้อมูลเหล่านี้เป็นรายละเอียดเพิ่มเติม การแยกออกเป็น ProductDetail ทำให้ Entity

มีความชัดเจนและสอดคล้องกับหลัก SRP

2. ความแตกต่างระหว่าง 1:1 และ 1:N

One-to-One หรือ 1:1 หมายถึงข้อมูลหนึ่งรายการมีความสัมพันธ์กับข้อมูลอีกหนึ่งรายการ ใน Lab นี้คือ Product กับ ProductDetail

ตัวอย่างเช่น Product หนึ่งรายการมี ProductDetail ได้หนึ่งชุด โดย

ProductDetail ใช้เก็บรายละเอียดเพิ่มเติม เช่น คำอธิบายสินค้า การรับประกัน น้ำหนัก และขนาด

ความสัมพันธ์แบบนี้เหมาะกับข้อมูลที่มีความสัมพันธ์กันแบบหนึ่งต่อหนึ่ง

และต้องการแยกข้อมูลบางส่วนออกจาก Entity หลักเพื่อให้โครงสร้างเป็นระเบียบ

One-to-Many หรือ

1:N หมายถึงข้อมูลหนึ่งรายการสามารถมีข้อมูลที่เกี่ยวข้องได้หลายรายการ

ใน Lab นี้คือ Product กับ Review

ตัวอย่างเช่น iPhone หนึ่งเครื่องสามารถมีลูกค้ารีวิวได้หลายคน ดังนั้น

Product หนึ่งรายการจึงสามารถมี Review ได้หลายรายการ

ความสัมพันธ์แบบนี้เหมาะกับข้อมูลที่สามารถเพิ่มจำนวนรายการได้เรื่อย ๆ

เช่น Product มีหลาย Review, Customer มีหลาย Order หรือ Department มีพนักงานหลายคน

สรุปง่าย ๆ

- 1:1 → หนึ่ง Product มี ProductDetail หนึ่งชุด
- 1:N → หนึ่ง Product มี Review ได้หลายรายการ
- 1:1 เหมาะกับข้อมูลเสริมที่มีได้เพียงหนึ่งชุด
- 1:N เหมาะกับข้อมูลหลักที่สามารถมีหลายรายการ

3. Strategy Pattern สำหรับคำนวณส่วนลด

Strategy Pattern

เป็นรูปแบบการออกแบบที่ใช้แยกวิธีการคำนวณที่แตกต่างกันออกจากกัน

ในระบบนี้มีส่วนลดหลายประเภท เช่น

- ไม่มีส่วนลด
- ส่วนลดสมาชิก 10%
- ส่วนลดเทศกาล 20%

แทนที่จะเขียนเงื่อนไขการคำนวณส่วนลดทั้งหมดรวมกันใน

Service ระบบจะแยกวิธีคำนวณแต่ละแบบออกเป็น Strategy

ข้อดีคือ หากในอนาคตต้องการเพิ่มส่วนลดประเภทใหม่ เช่น ส่วนลด VIP

หรือส่วนลดวันเกิด ก็สามารถเพิ่ม Strategy ใหม่ได้โดยไม่ต้องแก้ไขวิธีคำนวณของ

Strategy เดิม

ดังนั้น Strategy Pattern ช่วยให้โค้ดมีความยืดหยุ่นและสอดคล้องกับหลัก OCP และ DIP

4. Execution Flow: HTTP Request → Database

การทำงานของระบบแบ่งออกเป็นหลาย Layer โดยแต่ละ Layer มีหน้าที่ของตัวเอง เพื่อให้เป็นไปตามหลัก Layered Architecture และ SRP

ลำดับการทำงานคือ

Browser → Controller → Service → Repository → JPA/Hibernate → PostgreSQL

เมื่อ Database ทำงานเสร็จ ข้อมูลจะไหลกลับในทิศทางตรงกันข้าม

PostgreSQL → Repository → Service → Controller → Browser

Controller

Controller เป็นจุดแรกที่ได้รับ HTTP Request จากผู้ใช้ เช่น การเปิดหน้า Product

การเพิ่มสินค้า หรือการลบสินค้า

หน้าที่ของ Controller คือรับ Request และส่ง Response เท่านั้น ไม่ควรใส่ business logic จำนวนมากไว้ใน Controller

Service หลังจาก Controller รับ Request แล้ว จะส่งข้อมูลต่อไปให้ Service

Service มีหน้าที่จัดการ Business Logic เช่น การคำนวณส่วนลด การตรวจสอบข้อมูล

หรือการจัดการความสัมพันธ์ระหว่าง Product กับ ProductDetail

ดังนั้น Service จึงเป็นส่วนที่ตัดสินใจว่า Business Logic ของระบบต้องทำอะไร Repository

เมื่อ Service ต้องการอ่านหรือบันทึกข้อมูล จะเรียก Repository

Repository มีหน้าที่ติดต่อกับ Database ผ่าน Spring Data JPA โดย Service ไม่จำเป็นต้องเขียน SQL เอง JPA / Hibernate

JPA เป็นมาตรฐานสำหรับการเชื่อม Object ใน Java กับตารางใน Database

ส่วน Hibernate เป็น Implementation ที่ทำหน้าที่แปลงการทำงานของ Java Object

ให้กลายเป็น SQL แล้วส่งไปยัง PostgreSQL

ตัวอย่างเช่น เมื่อบันทึก Product Hibernate จะสร้าง SQL

สำหรับ INSERT ข้อมูลลงในตารางที่เกี่ยวข้อง PostgreSQL

PostgreSQL เป็น Database ที่เก็บข้อมูลจริง เช่น

- products
- product_details
- reviews

เมื่อ Database ทำงานเสร็จ ผลลัพธ์จะถูกส่งกลับขึ้นมาตาม Layer จนถึง Controller และ Controller ส่ง Response กลับไปให้ Browser

ส่วนที่ 2: Code + คำอธิบาย

1. Entity ทั้ง 3 ตัว

Product เป็น Entity หลักสำหรับเก็บข้อมูลสินค้า เช่น ชื่อสินค้า หมวดหมู่ ยี่ห้อ

จำนวนในคลัง ราคา และประเภทส่วนลด โดยมีความสัมพันธ์กับ ProductDetail แบบ One-to-One และมีความสัมพันธ์กับ Review แบบ One-to-Many

- o @Entity ใช้กำหนดว่า Product เป็น Entity ที่ JPA จัดการและเชื่อมกับ Database
 - o @Table(name = "products") กำหนดชื่อตารางเป็น products
 - o @Id กำหนด id เป็น Primary Key
 - o @GeneratedValue ให้ Database สร้างค่า ID อัตโนมัติ
 - o @OneToOne กำหนดความสัมพันธ์ 1:1 ระหว่าง Product กับ ProductDetail
 - o @JoinColumn(name = "detail_id") กำหนด Foreign Key detail_id ในตาราง Product
 - o @OneToMany กำหนดว่า Product หนึ่งรายการมี Review ได้หลายรายการ
- o mappedBy = "product" หมายความว่า Foreign Key ของความสัมพันธ์นี้อยู่ที่ Review ไม่ได้อยู่ที่ Product

ProductDetail ใช้เก็บข้อมูลรายละเอียดเพิ่มเติมของสินค้า เช่น description, warranty, weight, dimensions และประเทศที่ผลิต โดยเป็นความสัมพันธ์แบบ One-to-One กับ Product

- o @Entity ทำให้ ProductDetail เป็น Entity ของ JPA
- o @Id กำหนด Primary Key
- o @GeneratedValue สร้าง ID อัตโนมัติ
- o @OneToOne(mappedBy = "detail") ระบุว่า ProductDetail เป็นฝั่งตรงข้ามของความสัมพันธ์ และไม่ได้เป็นผู้ถือ Foreign Key
- o Foreign Key จะอยู่ที่ Product ใน detail_id

Review ใช้เก็บข้อมูลรีวิวสินค้า ได้แก่ reviewer, rating, comment และ reviewDate โดย Review เป็นฝั่ง Many ของความสัมพันธ์ Product กับ Review

- o @Entity ทำให้ Review เป็น Entity
- o @Id กำหนด Primary Key
- o @GeneratedValue สร้าง ID อัตโนมัติ
- o @ManyToOne หมายความว่า Review หลายรายการสามารถเป็นของ Product หนึ่งรายการ
- o @JoinColumn(name = "product_id") กำหนด Foreign Key product_id ในตาราง Review ซึ่งอ้างอิงไปยัง products.id

ดังนั้นความสัมพันธ์คือ

Product 1 : 1 ProductDetail และ Product 1 : N Review

โดย Foreign Key ของ 1:1 อยู่ที่ Product และ Foreign Key ของ 1:N อยู่ที่ Review ซึ่งเป็นไปตามหลักการออกแบบ Relationship ของ JPA

2. Service

ProductService มีหน้าที่จัดการ Business Logic ของระบบ เช่น การเพิ่ม แก้ไข ค้นหา และลบ Product โดย Service จะไม่ติดต่อ Database โดยตรง แต่จะเรียกผ่าน ProductRepository ลำดับการทำงานคือ

Controller → ProductService → ProductRepository → Database

การแยก Service ออกมาแบบนี้ช่วยให้แต่ละ Class มีหน้าที่ชัดเจนตามหลัก Single Responsibility Principle (SRP)

3. Controller

ProductController มีหน้าที่รับและตอบสนองต่อ HTTP Request จากผู้ใช้ เช่น

- o /products สำหรับแสดงรายการสินค้า
- o /products/add สำหรับแสดงฟอร์มเพิ่มสินค้า
- o /products/save สำหรับบันทึกสินค้า
- o /products/edit/{id} สำหรับแก้ไขสินค้า
- o /products/update/{id} สำหรับอัปเดตสินค้า
- o /products/delete/{id} สำหรับลบสินค้า

Controller จะไม่จัดการ Business Logic หรือ Database โดยตรง

แต่จะส่งงานต่อให้ ProductService

ดังนั้นหน้าที่ของ Controller คือ รับ Request → เรียก Service → ส่ง Response

4. Constructor Injection

ใน ProductService จะมี Dependency คือ ProductRepository ส่วน ProductController จะมี Dependency คือ ProductService

เราใช้ Constructor Injection เพื่อให้ Spring เป็นผู้ส่ง Dependency เข้ามาทาง Constructor แทนที่จะสร้าง Object ขึ้นมาเอง

ตัวอย่างการทำงานคือ

Spring → สร้าง ProductRepository → ส่งให้ ProductService

และ

Spring → สร้าง ProductService → ส่งให้ ProductController

ข้อดีของ Constructor Injection คือ Dependency ของแต่ละ Class ชัดเจน

สามารถใช้ final ได้ง่าย ทดสอบ Unit Test ได้ง่าย และลดการผูกกับ Implementation โดยตรง

จึงสอดคล้องกับ Dependency Inversion Principle (DIP) ที่ให้ Class พึ่งพา Abstraction เช่น Repository Interface แทนการพึ่งพา Implementation โดยตรง

Create

Product Shop

Lab 8 — 1:1 & 1:N Relationship

+ เพิ่มสินค้าใหม่

ข้อมูลสินค้า (Product)

ชื่อสินค้า

iphone 15 673380057-5 SEC1

หมวดหมู่

Phone

มีชื่อ

Pineapple

จำนวนในคลัง

15

ราคาปกติ (บาท)

15000

ประเภทส่วนลด

ไม่มีส่วนลด

ข้อมูลเสริมสินค้า (ProductDetail) — ความสัมพันธ์ 1:1

รายละเอียดสินค้า

ของดี

Read

Product Shop

Lab 8 — 1:1 & 1:N Relationship

+ เพิ่มสินค้าใหม่

#	ชื่อสินค้า	หมวดหมู่	มีชื่อ	คลัง	ราคา (บาท)	ส่วนลด	ราคาสุทธิ	รับประกัน (1:1)	น้ำหนัก (1:1)	รีวิว (1:N)	จัดการ
1	iphone 15 673380057-5 SEC1	Phone	Pineapple	15	15000.00	NONE		1 year	1.61	0 รีวิว	แก้ไขลบ

Update

แก้ไขสินค้า

ข้อมูลสินค้า (Product)

ชื่อสินค้า

iphone 15 673380057-5 SEC1

หมวดหมู่

Phone

ยี่ห้อ

Pineapple

จำนวนในคลัง

15

ราคาปกติ (บาท)

15000.0

ประเภทส่วนลด

ไม่มีส่วนลด

ข้อมูลเสริมสินค้า (ProductDetail) — ความสัมพันธ์ 1:1

รายละเอียดสินค้า

ทองดี

Delete

ยืนยันการลบสินค้า

คุณต้องการลบสินค้านี้ใช่ไหม? การดำเนินการนี้ไม่สามารถย้อนกลับได้

ชื่อสินค้า: Iphone 15 673380057-5 SEC1

หมวดหมู่: Phone

ยี่ห้อ: Pineapple

ราคา: 15000.00 บาท

รายละเอียด: ทองดี

รับประกัน: 1 year

ข้อมูลเสริม (ProductDetail) จะถูกลบพร้อมกันด้วย เนื่องจาก CascadeType.ALL

ยืนยันลบ

ยกเลิก

Product Shop

Lab 8 — 1:1 & 1:N Relationship

+ เพิ่มสินค้าใหม่

#	ชื่อสินค้า	หมวดหมู่	ยี่ห้อ	คลัง	ราคา (บาท)	ส่วนลด	ราคาสุทธิ	รับประกัน (1:1)	น้ำหนัก (1:1)	รีวิว (1:N)	จัดการ
ยังไม่มีสินค้า — กด "+ เพิ่มสินค้าใหม่" เพื่อเริ่มต้น											



Tables (3)
product_details
Columns (6)
id bigint
description character varying(255)
dimensions character varying(255)
manufactured_country character varying(255)
warranty character varying(255)
weight double precision
> Constraints
> Indexes
> RLS Policies
> Rules
> Triggers
products
Columns (9)
id bigint
brand character varying(255)
category character varying(255)
discount_type character varying(255)
name character varying(255)
price double precision
stock integer
detail_id bigint
discounted_price double precision
> Constraints
> Indexes
> RLS Policies
> Rules
> Triggers
reviews
Columns (6)
id bigint
comment character varying(255)
rating integer
review_date date
reviewer character varying(255)
product_id bigint